

Task constructor (from OpenMP 2.5)

```
int fib(int n)
{
    int i, j;
    if (n<2) return n;
    else
    {
        #pragma omp task shared(i) firstprivate(n)
        i=fib(n-1);

        #pragma omp task shared(j) firstprivate(n)
        j=fib(n-2);

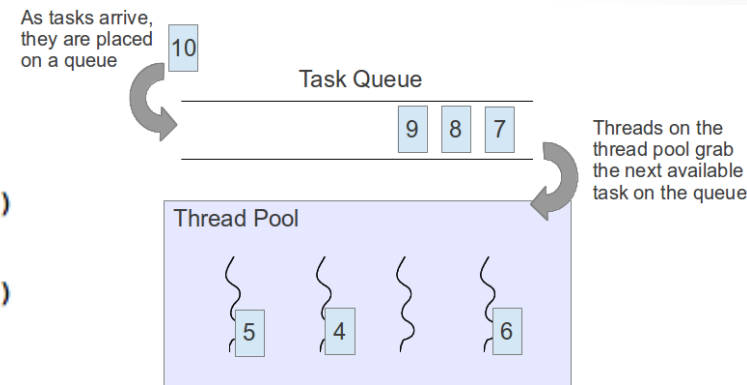
        #pragma omp taskwait
        return i+j;
    }
}

int main()
{
    int n = 10;

    omp_set_num_threads(4);

    #pragma omp parallel shared(n)
    {
        #pragma omp single
        printf ("fib(%d) = %d\n", n, fib(n));
    }
}
```

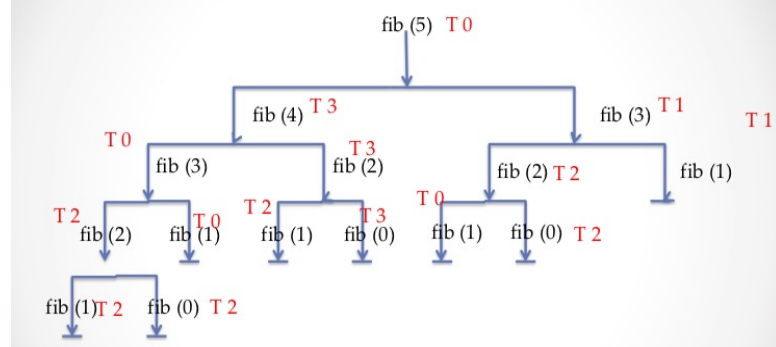
➤ Execution model:



- The **taskwait** directive ensures that the tasks generated are completed before the return statements.

- Only one thread executes the **single** directive, and hence the call to fib(n),
- **All four threads** will participate in executing the tasks generated.

Execution Tree Exemplified



Data Sharing

- What happens to variables in parallel regions?
 - Variables declared **inside** are **local** to each thread; => thread local stack or register
 - Variables declared **outside** are **shared** => main memory accessed by all threads
- Data sharing clauses:
 - **private(varlist)** => each variable in *varlist* becomes private to each thread, initial values not specified.
 - **firstprivate(varlist)** => Same as private, but variables are initialised with the value outside the region.
 - **lastprivate(varlist)** => same as private, but the final value is the last loop iteration's value.
 - **reduction (op:var)** => same as *lastprivate*, but the final value is the result of reduction of private values using the operator "**op**".
- ~~• Directives for data sharing:~~
 - ~~◦ **#pragma omp threadlocal** => each thread gets a local copy of the value.~~
 - ~~◦ **copyin** clause copies the values from thread master to the others threads.~~

Synchronization Constructs:

- Critical regions (executed in mutual exclusion):
 - `#pragma omp critical [name]`
 `someFunctionCall();`
 - Restricts the execution of the associated structured blocks to a single thread at a time;
 - Works inter-teams (i.e., global lock)
 - An optional name may be used to identify the critical region.
- Atomic Operations (fine-grain synchronization):
 - `#pragma omp atomic`
 `A[i] += x;`
 - The memory in will be updated atomically. It does not make the entire statement atomic; only the memory update is atomic.
 - A compiler might use special hardware instructions for **better** performance than when using **critical**.

Synchronization Constructs:

- Solving data races with critical VS atomic

%r12 contains a pointer to a shared memory position

```

5  int main(){
6      double result={0};
7
8      #pragma omp parallel for shared(result)
9      for(int i=0; i<1000000;i++) {
10         result+=sin(i);
11     }
12     printf("%f",result);
13 }

```

Data race

```

.L4: vxorpd %xmm0, %xmm0, %xmm0
      vcvtsi2sdl %ebx, %xmm0, %xmm0 ; convert to double
      call sin
      vmovsd 8(%rsp,%r12), %xmm1 ; load result into %xmm1
      vaddsd %xmm0, %xmm1, %xmm1 ; add
      vmovsd %xmm1, 8(%rsp,%r12) ; update result
      ...
      jne .L4

```

- Critical:** coarse-grain and pessimist

```

call GOMP_critical_start
vmovsd 8(%rsp,%r12), %xmm1 ; load result into %xmm1
vaddsd %xmm0, %xmm1, %xmm1
vmovsd %xmm1, 8(%rsp,%r12) ; update results
call GOMP_critical_end

```

Mutual exclusion: ensures that only one thread executes the critical region (e.g., blocks the calling thread if another thread is still executing the region)

- Atomic:** fine-grain and optimistic (note: simplified code)

```

.L5: vmovsd (%r12), %xmm1
      vaddsd %xmm0, %xmm1, %xmm1
      ...
      lock cmpxchgq %xmm1, (%r12) ; atomic comp & exchange
      cmpq %rax, %rdx
      jne .L5 ; repeat if update failed

```

Repeat until successful update

Avoid/reduce synchronisation

- Reduction of multiple values (in parallel):

```
sum = 0;
#pragma omp parallel for reduction(+:sum)
for(int i = 0; i < 100; i++) {
    sum += array[i];
}
```

Each thread owns a private copy of sum
At the end of the loop execution private
values are combined

- Thread reuse across parallel regions

```
#pragma omp parallel {
    #pragma omp for
    for(int i = 0; i < 100; i++)
        ...
    #pragma omp for
    for(int j = 0; j < 100; j++)
        ...
}
```

There is an implicit
barrier at the end of
each for

Environment variables

- **OMP_SCHEDULE**
 - sets the *run-sched-var* ICV for the runtime schedule type and chunk size. It can be set to any of the valid OpenMP schedule types (i.e., **static**, **dynamic**, **guided**, and **auto**).
- **OMP_NUM_THREADS**
 - sets the *nthreads-var* ICV for the number of threads to use for parallel regions.
- **OMP_DYNAMIC**
 - sets the *dyn-var* ICV for the dynamic adjustment of threads to use for **parallel** regions.
- **OMP_NESTED**
 - sets the *nest-var* ICV to enable or to disable nested parallelism.
- **OMP_STACKSIZE**
 - sets the *stacksize-var* ICV that specifies the size of the stack for threads created by the OpenMP implementation.
- **OMP_WAIT_POLICY**
 - sets the *wait-policy-var* ICV that controls the desired behavior of waiting threads.
- **OMP_MAX_ACTIVE_LEVELS**
 - sets the *max-active-levels-var* ICV that controls the maximum number of nested active parallel regions.
- **OMP_THREAD_LIMIT**
 - sets the *thread-limit-var* ICV that controls the maximum number of threads participating in the OpenMP program.

OpenMP Routines

- **omp_set_num_threads / omp_get_num_threads**
- omp_get_max_threads
- **omp_get_thread_num**
- omp_get_num_procs.
- omp_in_parallel.
- omp_set_dynamic / omp_get_dynamic.
- omp_set_nested / omp_get_nested.
- omp_set_schedule / omp_get_schedule
- omp_get_thread_limit.
- omp_set_max_active_levels / omp_get_max_active_levels
- omp_get_level.
- omp_get_ancestor_thread_num.
- omp_get_team_size.
- omp_get_active_level
- **Locks**
 - void omp_init_lock(omp_lock_t *lock);
 - void omp_destroy_lock(omp_lock_t *lock);
 - void omp_set_lock(omp_lock_t *lock);
 - void omp_unset_lock(omp_lock_t *lock);
 - int omp_test_lock(omp_lock_t *lock);
- **Timers**
 - **double omp_get_wtime(void);**
 - double omp_get_wtick(void);